Robot Imitation Learning: From BC and DAgger to Real-Time VLA Execution

Supervision, Generative Action Chunks, Multimodal Policies and Real-Time Execution

2026-10-30

Contents

1. Imitation Learning Foundations: BC and DAgger 2
2. Action Chunking and Generative Imitation Learning 2
3. From Imitation Learning to VLA 3
4. Real-Time Execution: From RTC to Future-State Compensation 4
References and Further Reading 5

Course home ·简体中文 ·PDF

Item	Details
Topic	Imitation Learning
Revision	2026-10-30
Prerequisites	Deep learning foundations and introductory robot control
License	CC BY-NC-SA 4.0 ·Xiong Zeng and contributors

1. Imitation Learning Foundations: BC and DAgger

1.1 A unified learning problem

Let s_t denote the partially observed physical state, o_t exteroceptive observations, q_t proprioception, and g the task. A history-conditioned policy uses $h_t = (o_{\le t}, q_{\le t}, g)$ to predict $a_t \sim \pi_\theta(\cdot \mid h_t)$. The actuator command $u_t = \text{Controller}(a_t, q_t)$ may differ from the model output due to limits, filters, and low-level control.

Define action semantics before the network: joint positions, increments, velocities, end-effector poses, and hand targets are different labels. Units, frames, timestamps, control modes, and training-set normalization must remain consistent at deployment.

1.2 Behavior Cloning and its limits

With expert dataset $\mathcal{D}_E = \{(h_i, a_i^E)\}$, deterministic BC minimizes $\mathbb{E}_{\mathcal{D}}[\|f_\theta(h) - a^E\|_W^2]$. This is natural for approximately unimodal conditional action distributions. If there are two distinct valid routes, their conditional mean can be unsafe; latent-variable and generative policies instead represent multimodality.

BC also faces **covariate shift**: the learner visits states outside the expert occupancy distribution after errors. Classical finite-horizon analyses may exhibit $\mathcal{O}(T^2\varepsilon)$ worst-case accumulated cost scaling. Such bounds are not direct predictors of success rates from continuous-action MSE.

1.3 DAgger: supervise states encountered by the learner

DAgger is a **data collection and update protocol**, not an action-head architecture. At round k: roll out the current or mixed policy → encounter new states → ask the expert to label those states → aggregate $\mathcal{D}_{k+1} = \mathcal{D}_k \cup \Delta\mathcal{D}_k$ → retrain. A mixed rollout selects which policy controls the system; it need not average their action vectors.

In robotics, labeling every visited state can be expensive. Log executed commands separately from expert targets. Unsafe or irrecoverable states require stopping/resetting rather than forcing an expert action.

1.4 Intervention learning: HG-DAgger, IWR, and RaC

Method	Main idea	Question answered
DAgger	Expert-label learner-visited states	How do we mitigate shift?
HG-DAgger	Humans intervene at critical times	How can we reduce labeling effort?
IWR	Upweight intervention data in supervised updates	How can rare corrections matter?
RaC	Separate Recovery from Correction	How should the robot return to a recoverable situation?

Distinct roles: corrective data collection improves coverage; a generative action head improves the representation of multiple valid actions. Record intervention times, contact phases, executed commands, and expert labels.

2. Action Chunking and Generative Imitation Learning

2.1 Prediction horizon versus execution horizon

At time t, predict $A_t = (a_{t|t}, \dots, a_{t+H-1|t})$, where H is the prediction horizon. Execute $K \le H$ actions before re-observing/replanning. A larger K saves inference calls but lengthens the open-loop interval; H and K must not be conflated.

2.2 ACT: Transformer + conditional VAE

ACT conditions a Transformer on multi-view images and proprioception. At training time an encoder infers a latent z from expert actions; the decoder predicts a complete chunk $\hat{A}_t = f_\theta(h_t, z)$. A typical objective is $\mathcal{L}_{\text{ACT}} = \mathcal{L}_{\text{masked-}L_1} + \beta\,\mathrm{KL}(q_\phi(z \mid q_t, A_t^E) \,\|\, \mathcal{N}(0, I))$; padding requires masks. At inference, future expert actions are unavailable, and original ACT commonly uses the prior mean $z = 0$.

Temporal Ensembling combines predictions from multiple past query times that target the **same physical execution instant**. It is not ordinary smoothing across adjacent time steps and can suppress desired rapid switches during contact.

2.3 Diffusion Policy and Flow Matching

Diffusion Policy learns a denoiser on noisy action chunks: $\mathcal{L}_{\text{diff}} = \mathbb{E}\|\varepsilon - \varepsilon_\theta(A_{\text{noisy}}, n, h)\|^2$. At inference it samples via iterative denoising, enabling multimodal trajectories at a computational cost.

Flow Matching interpolates between noise A_0 and expert actions A_1: $A_\tau = (1-\tau)A_0 + \tau A_1$. It minimizes $\mathcal{L}_{\text{FM}} = \mathbb{E}\|v_\theta(A_\tau, \tau, h) - (A_1 - A_0)\|^2$ and integrates $\mathrm{d}A/\mathrm{d}\tau = v_\theta(A, \tau, h)$ at inference. Neither generative objective by itself ensures smooth execution on the robot's wall clock.

2.4 RDP: high-frequency tactile feedback within a chunk

Reactive Diffusion Policy (RDP) combines a low-frequency **slow latent diffusion** policy with a high-frequency **fast tactile-feedback** module. Its motivation differs from faster chunk generation: it addresses rapid physical correction *while the current chunk executes*. It is a useful hierarchical-control example for dexterous screwing, insertion, and slip mitigation, not a universal replacement for all policies.

3. From Imitation Learning to VLA

3.1 A VLA does not replace BC

A vision-language-action model conditions actions on images, proprioception, and language: $z_t = F(I_{\le t}, q_{\le t}, \text{language})$ followed by an action decoder. Robot-action supervised pretraining and downstream supervised fine-tuning remain forms of BC in a broad sense; DAgger determines how to obtain additional expert supervision on learner-visited states.

3.2 Two common output interfaces

Output	Learning approach	Key trade-off
Discrete action tokens	Autoregressive cross-entropy (e.g., OpenVLA family)	Language-model compatibility versus quantization and token throughput
Continuous action chunks	Diffusion / Flow Matching (e.g., π0 family)	High-dimensional precision versus sampling and execution latency

FAST / FAST+ (optional): frequency-domain compression for action tokenization. It shows why tokenization is not synonymous with per-joint scalar discretization. It addresses **representation**, not chunk synchronization.

3.3 Dataset and embodiment contracts

Training records should specify (images, proprioception, instruction, embodiment-ID, expert-action-chunk, mask, timestamps). Zero-padding alone cannot unify semantically different joints, frames, or command rates. Distinguish initial robot-action training from adaptation of a pretrained VLA, including exactly which modules are updated.

Levels: BC / DAgger govern supervision; ACT / Diffusion / Flow govern conditional action distributions; VLA governs multimodal task conditioning; RTC / VLASH govern execution in physical time.

4. Real-Time Execution: From RTC to Future-State Compensation

4.1 Two clocks and three lengths

The robot runs at a control interval Δt_c, whereas the policy has end-to-end latency T_{lat}, including camera capture, preprocessing, queueing, inference, and transport. Let $d = \left\lceil \frac{T_{\text{lat}}}{\Delta t_c} \right\rceil$. Distinguish **prediction horizon** H, **executed horizon** K, **and latency in control steps** d. For $\Delta t_c = 20\,\text{ms}$ and $T_{\text{lat}} = 110\,\text{ms}$, $d = 6$.

Synchronous inference can stall. Naive asynchronous execution keeps the old chunk running but may create abrupt chunk switches. There are two separable mismatches: **chunk-prefix continuity** and **stale conditioning information**.

4.2 RTC and Training-Time RTC: committed action prefixes

Inference-Time RTC conditions new-chunk generation on previously committed actions, using inpainting/guidance for consistency, at possible inference overhead. Training-Time RTC learns prefix-conditioned generation during training to reduce the need for inference-time guidance, at a cost of retraining and adaptation to delay ranges. These mechanisms principally target **boundary continuity**, not necessarily the true observation at execution time.

4.3 VLASH: align state conditioning with execution time

While the new chunk is being computed, the robot continues executing A_{old}; therefore the initial q_t can be stale by execution time. VLASH rolls proprioception forward:

$$\hat{q}(t+\Delta) = \text{RollForward}(q_t, A_{\text{old}}, \Delta)$$

$$A_{\text{new}} = \pi_\theta(o_t, \hat{q}(t+\Delta))$$

The available image o_t may **still be stale**. VLASH mainly corrects the temporal mismatch in proprioception; it should not be described as fully predicting future visual observations.

Crucial nuance: VLASH is not merely training-free rollforward. Its **Temporal-Offset Fine-Tuning** samples offset δ and trains with $(o_t, q_{t+\delta})$ to predict expert actions $A^E_{t+\delta:t+\delta+H-1}$. Rollforward is an inference mechanism; offset fine-tuning teaches the model to actually use future proprioceptive inputs. Both parts matter in interpreting the paper.

4.4 FutureRTC: include anticipated visual features

FutureRTC goes beyond proprioception: using motion-aware feature transport/reconstruction it predicts execution-time visual representations, adds proprioceptive-state correction, and uses policy-consistency training. Its predicted visual features are **not** equivalent to future ground-truth images. Unexpected contacts, novel occlusions, and external changes remain difficult.

4.5 RDP and RTR: different real-time bottlenecks

RDP focuses on high-rate tactile feedback **inside** a chunk. RTR (optional reading) explores high-frequency action reuse/refinement and smoothness. These address partially overlapping but distinct system problems and should not be ranked using success rates from incomparable benchmarks.

Method	Bottleneck	Main mechanism	Important caveat
Temporal Ensembling	Conflicting overlapping predictions	Fuse predictions for the same time	Excessive smoothing at contacts
Inference-Time RTC	Discontinuous chunk switching	Committed-prefix guidance	Inference overhead
Training-Time RTC	Guidance cost	Learn prefix conditioning at training time	Retraining / delay generalization

Method	Bottleneck	Main mechanism	Important caveat
VLASH	Stale proprioception	State rollforward + offset fine-tuning	Model error; stale images
FutureRTC	Stale visual and proprio inputs	Anticipated visual/state features	Forecast error, external events
RDP	Slow in-chunk contact correction	Slow–Fast tactile closed loop	Tactile sensing and data needs
RTR	High-frequency action jitter	Reuse-then-Refine	Different objective from RTC

4.6 Putting the layers together

1. Validate low-level command semantics, actuator limits, timestamps, and tracking stability before changing the learning algorithm.
2. Choose H, K, Δt_c; measure **end-to-end latency and tail latency**, not GPU forward time alone.
3. If chunk switching jumps, investigate RTC. If model conditioning is stale, investigate VLASH/FutureRTC. If contact responses are too slow during chunk execution, investigate RDP-like high-rate feedback.
4. Evaluate continuity, force peaks, collisions, latency distribution, recovery after intervention, and task success independently. Combining methods requires empirical validation.

Takeaway: imitation learning is not just action prediction. A deployable system requires coverage of the relevant states, an adequate conditional action model, multimodal task conditioning, and safe feedback execution under the robot's physical clock.

References and Further Reading

1. Ross et al. **DAgger**, 2011. https://arxiv.org/abs/1011.0686
2. Kelly et al. **HG-DAgger**, 2018. https://arxiv.org/abs/1810.02890
3. Mandlekar et al. **Interactive Interventions / IWR**. https://sites.google.com/stanford.edu/iwr
4. Zhao et al. **ACT**, 2023. https://arxiv.org/abs/2304.13705
5. Chi et al. **Diffusion Policy**, 2023. https://arxiv.org/abs/2303.04137
6. Lipman et al. **Flow Matching**, 2023. https://arxiv.org/abs/2210.02747
7. Kim et al. **OpenVLA**, 2024. https://arxiv.org/abs/2406.09246
8. Black et al. **π0**, 2024. https://www.physicalintelligence.company/blog/pi0
9. Perreault et al. **FAST**, 2025. https://www.roboticsproceedings.org/rss21/p012.pdf
10. **Real-Time Chunking (RTC)**. https://arxiv.org/search/?query=Real-Time+Chunking&searchtype=all
11. **VLASH**, 2025. https://arxiv.org/abs/2512.01031
12. **FutureRTC**, 2026. https://arxiv.org/abs/2607.24008
13. Xue et al. **Reactive Diffusion Policy**, 2025. https://arxiv.org/abs/2503.02881
14. **RTR**, further reading. https://proceedings.mlr.press/v306/wang26lj.html

Licensing: Third-party papers and figures retain their own licenses.